

GPT-OSS_PT_s4e8

April 16, 2026

1 Previsão Binária de Cogumelos Venenosos

2 Etapa 1: Carregamento dos Dados

Nesta primeira etapa do notebook, vamos **carregar** os arquivos `train.csv` e `test.csv` em DataFrames do pandas. O objetivo é ter as duas bases prontas para as próximas etapas de análise exploratória e pré-processamento.

2.1 Passos que serão executados

1. **Leitura dos arquivos:** utilizaremos `pd.read_csv()` apontando para o diretório onde os arquivos estão armazenados. Se os arquivos estiverem na raiz do projeto, basta usar os nomes `train.csv` e `test.csv`.
2. **Verificação de dimensões:** imprimiremos `df.shape` para cada conjunto, garantindo que o número de linhas e colunas corresponde ao esperado.
3. **Checagem de valores ausentes:** aplicaremos `df.isnull().sum()` para identificar colunas com missing values. Isso nos ajudará a decidir se precisamos de imputação ou remoção.
4. **Análise de tipos de dados:** com `df.dtypes` veremos quais colunas são numéricas, categóricas ou de texto. No Kaggle, a maioria das colunas são categóricas codificadas por letras, então precisaremos tratá-las adequadamente nas etapas seguintes.

2.2 Observações importantes

- **Não alteraremos a coluna alvo classe** neste momento. Ela será mantida em seu formato original (c/v) para que possamos mapear para 0/1 apenas quando for necessário treinar o modelo.
- **Não haverá escrita de arquivos** nesta etapa. Apenas carregaremos os dados na memória para uso posterior.
- **Separação de dados:** manteremos `train_df` e `test_df` como objetos distintos para evitar vazamento de informação entre treino e teste.

2.3 Próximos passos

Após o carregamento, avançaremos para a **Análise Exploratória dos Dados (EDA)**, onde examinaremos a distribuição das classes, a presença de outliers e a cardinalidade das variáveis categóricas. Essa etapa nos guiará na escolha de técnicas de codificação e na definição de estratégias de imputação.

(Esta célula serve apenas como documentação e orientação. O código real de leitura será inserido na próxima célula de código.)

```
[1]: import pandas as pd

# Carregar os arquivos CSV
train_df = pd.read_csv('train.csv')
test_df = pd.read_csv('test.csv')

# Exibir dimensões dos DataFrames
print('Dimensões do conjunto de treino:', train_df.shape)
print('Dimensões do conjunto de teste:', test_df.shape)

# Verificar valores ausentes
print('\nValores ausentes no treino:')
print(train_df.isnull().sum())

print('\nValores ausentes no teste:')
print(test_df.isnull().sum())

# Exibir tipos de dados
print('\nTipos de dados do conjunto de treino:')
print(train_df.dtypes)
```

Dimensões do conjunto de treino: (3116945, 22)

Dimensões do conjunto de teste: (2077964, 21)

Valores ausentes no treino:

id	0
classe	0
diametro_chapeu	4
formato_chapeu	40
superficie_chapeu	671023
cor_chapeu	12
machuca_ou_sangra	8
fixacao_lamela	523936
espacamento_lamela	1258435
cor_lamela	57
altura_estipe	0
largura_estipe	0
raiz_estipe	2757023
superficie_estipe	1980861
cor_estipe	38
tipo_veu	2957493
cor_veu	2740947
tem_anel	24
tipo_anel	128880
cor_impressao_esporo	2849682

```

habitat          45
estacao          0
dtype: int64

```

Valores ausentes no teste:

```

id              0
diametro_chapeu 7
formato_chapeu  31
superficie_chapeu 446904
cor_chapeu      13
machuca_ou_sangra 10
fixacao_lamela  349821
espacamento_lamela 839595
cor_lamela      49
altura_estipe   1
largura_estipe  0
raiz_estipe     1838012
superficie_estipe 1321488
cor_estipe      21
tipo_veu        1971545
cor_veu         1826124
tem_anel        19
tipo_anel       86195
cor_impressao_esporo 1899617
habitat         25
estacao         0
dtype: int64

```

Tipos de dados do conjunto de treino:

```

id              int64
classe          object
diametro_chapeu float64
formato_chapeu  object
superficie_chapeu object
cor_chapeu      object
machuca_ou_sangra object
fixacao_lamela  object
espacamento_lamela object
cor_lamela      object
altura_estipe   float64
largura_estipe  float64
raiz_estipe     object
superficie_estipe object
cor_estipe      object
tipo_veu        object
cor_veu         object
tem_anel        object
tipo_anel       object

```

```
cor_impressao_esporo    object
habitat                 object
estacao                 object
dtype: object
```

2.4 Etapa 2: Análise Exploratória dos Dados (EDA)

2.4.1 Visão geral da execução anterior

Na etapa anterior carregamos os arquivos `train.csv` e `test.csv` em dois DataFrames, `train_df` e `test_df`. Verificamos as dimensões, a presença de valores ausentes e os tipos de dados de cada coluna. Observamos que a maioria das variáveis são categóricas codificadas por letras, enquanto algumas possuem valores numéricos (ex.: `diametro_chapeu`, `altura_estipe`). Também identificamos que há valores ausentes em várias colunas, principalmente nas que representam características físicas do cogumelo.

2.4.2 Objetivo desta etapa

A Análise Exploratória dos Dados (EDA) tem como finalidade:

1. **Entender a distribuição das classes (classe)**, verificando a proporção entre cogumelos comestíveis (c) e venenosos (v). Isso nos ajudará a decidir se precisamos de técnicas de balanceamento.
2. **Explorar a cardinalidade das variáveis categóricas** – quantos níveis distintas cada coluna possui e se há níveis raros que podem ser consolidados.
3. **Detectar outliers e valores extremos** nas colunas numéricas, usando histogramas ou box-plots. Embora o conjunto seja composto principalmente por categorias, as colunas numéricas podem conter erros de digitação.
4. **Analisar a correlação entre variáveis** (para variáveis numéricas) e a correlação entre categorias e a classe alvo, usando métricas como Cramér's V ou chi-quadrado.
5. **Verificar a presença de valores ausentes** em cada coluna e decidir estratégias de imputação ou remoção.

2.4.3 Próximos passos após a EDA

Com os insights obtidos, passaremos para a etapa de pré-processamento, onde:

- Tratar os valores ausentes (imputação ou exclusão).
- Codificar as variáveis categóricas (LabelEncoder, OneHotEncoder ou Target Encoding).
- Mapear a coluna alvo `classe` para valores numéricos (0/1).
- Normalizar ou padronizar as colunas numéricas, se necessário.

Essas decisões serão fundamentadas nos resultados da EDA, garantindo que o modelo receba dados limpos e bem preparados.

(Esta célula contém apenas a documentação e a descrição do que será feito na etapa de EDA. O código real será inserido em células de código subsequentes.)

```
[2]: # Etapa 2: Análise Exploratória dos Dados (EDA)
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Distribuição da classe alvo
classe_counts = train_df['classe'].value_counts(dropna=False)
print('Distribuição da classe alvo:')
print(classe_counts)
print('\nProporção (c vs v):')
print(classe_counts / classe_counts.sum())

# 2. Cardinalidade das colunas
cardinality = train_df.nunique(dropna=False)
print('\nCardinalidade das colunas:')
print(cardinality.sort_values(ascending=False))

# 3. Top 5 valores mais frequentes em colunas categóricas
categorical_cols = train_df.select_dtypes(include=['object']).columns
print('\nTop 5 valores por coluna categórica:')
for col in categorical_cols:
    top5 = train_df[col].value_counts(dropna=False).head(5)
    print(f'\n{col}:')
    print(top5)

# 4. Estatísticas resumidas das colunas numéricas
numeric_cols = train_df.select_dtypes(include=['number']).columns
print('\nEstatísticas resumidas das colunas numéricas:')
print(train_df[numeric_cols].describe().transpose())

# 5. Correlação entre variáveis numéricas
if len(numeric_cols) > 1:
    corr = train_df[numeric_cols].corr()
    plt.figure(figsize=(8, 6))
    sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', square=True)
    plt.title('Matriz de Correlação (colunas numéricas)')
    plt.tight_layout()
    plt.show()
else:
    print('Não há colunas numéricas suficientes para calcular correlação.')
```

Distribuição da classe alvo:

```
classe
v    1705396
c    1411549
Name: count, dtype: int64
```

Proporção (c vs v):

```
classe
```

```
v    0.547137
c    0.452863
Name: count, dtype: float64
```

Cardinalidade das colunas:

```
id                3116945
largura_estipe    5836
diametro_chapeu   3914
altura_estipe     2749
superficie_chapeu 84
cor_chapeu        79
fixacao_lamela    79
formato_chapeu    75
cor_lamela        64
superficie_estipe 61
cor_estipe        60
habitat          53
espacamento_lamela 49
tipo_anel         41
raiz_estipe       39
cor_impressao_esporo 33
machuca_ou_sangra 27
cor_veu           25
tem_anel          24
tipo_veu          23
estacao           4
classe            2
dtype: int64
```

Top 5 valores por coluna categórica:

classe:

classe

```
v    1705396
c    1411549
Name: count, dtype: int64
```

formato_chapeu:

formato_chapeu

```
x    1436026
f     676238
s     365146
b     318646
o     108835
Name: count, dtype: int64
```

superficie_chapeu:

superficie_chapeu

```
NaN      671023
t        460777
s        384970
y        327826
h        284460
Name: count, dtype: int64
```

```
cor_chapeu:
cor_chapeu
n      1359542
y      386627
w      379442
g      210825
e      197290
Name: count, dtype: int64
```

```
machuca_ou_sangra:
machuca_ou_sangra
f      2569743
t      547085
w         14
c         11
h          9
Name: count, dtype: int64
```

```
fixacao_lamela:
fixacao_lamela
a      646034
d      589236
NaN     523936
x      360878
e      301858
Name: count, dtype: int64
```

```
espacamento_lamela:
espacamento_lamela
c      1331054
NaN     1258435
d      407932
f      119380
e         24
Name: count, dtype: int64
```

```
cor_lamela:
cor_lamela
w      931538
n      543386
y      469464
```

```
p    343626
g    212164
Name: count, dtype: int64
```

```
raiz_estipe:
raiz_estipe
NaN    2757023
b      165801
s      116946
r       47803
c       28592
Name: count, dtype: int64
```

```
superficie_estipe:
superficie_estipe
NaN    1980861
s      327610
y      255500
i      224346
t      147974
Name: count, dtype: int64
```

```
cor_estipe:
cor_estipe
w    1196637
n    1003464
y     373971
g     132019
o     111541
Name: count, dtype: int64
```

```
tipo_veu:
tipo_veu
NaN    2957493
u     159373
w         11
a          9
e          8
Name: count, dtype: int64
```

```
cor_veu:
cor_veu
NaN    2740947
w     279070
y     30473
n     30039
u     14026
Name: count, dtype: int64
```



```
tem_anel:
tem_anel
f      2368820
t      747982
NaN      24
r      16
h      13
Name: count, dtype: int64
```

```
tipo_anel:
tipo_anel
f      2477170
NaN     128880
e      120006
z      113780
l      73443
Name: count, dtype: int64
```

```
cor_impressao_esporo:
cor_impressao_esporo
NaN     2849682
k      107310
p      68237
w      50173
n      22646
Name: count, dtype: int64
```

```
habitat:
habitat
d     2177573
g     454908
l     171892
m     150969
h     120137
Name: count, dtype: int64
```

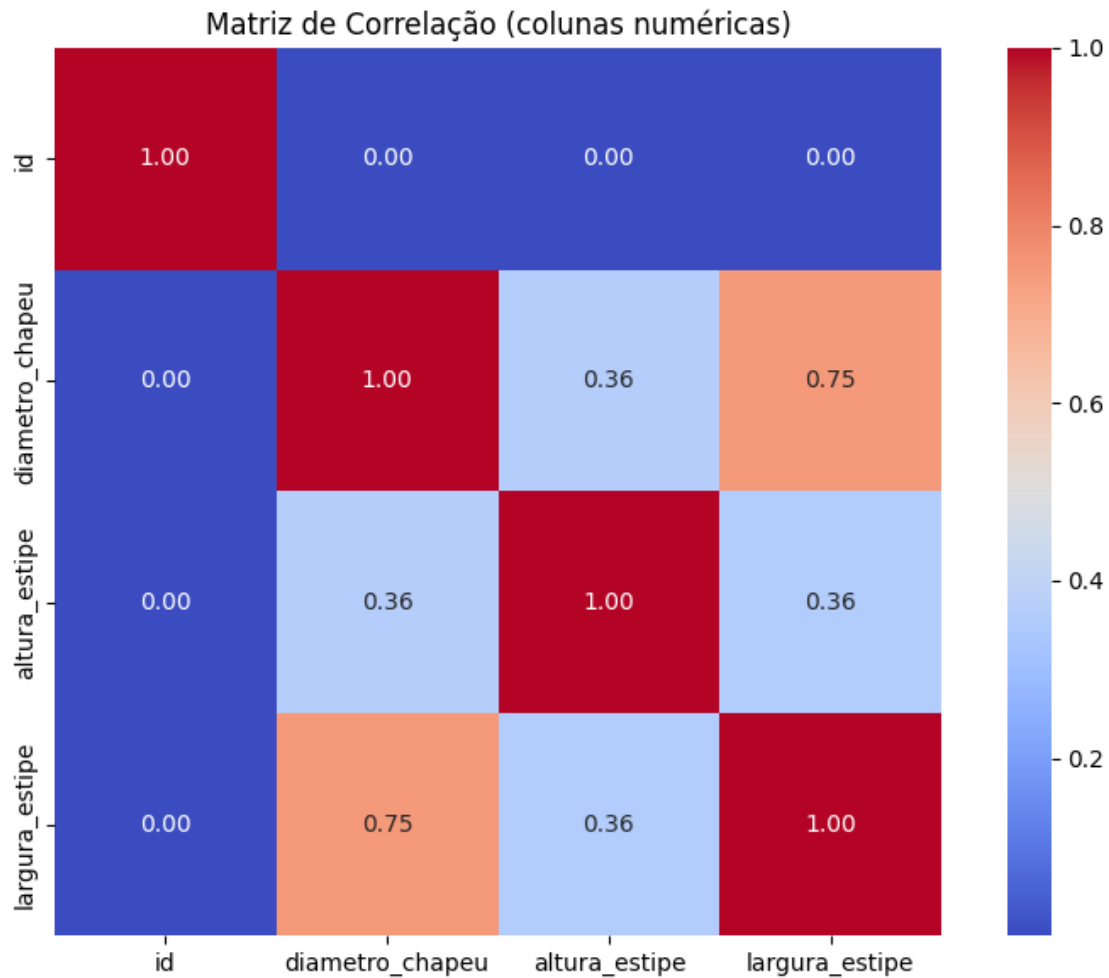
```
estacao:
estacao
a     1543321
u     1153588
w     278189
s     141847
Name: count, dtype: int64
```

Estatísticas resumidas das colunas numéricas:

	count	mean	std	min	25%	\
id	3116945.0	1.558472e+06	899784.661737	0.00	779236.00	

diametro_chapeu	3116941.0	6.309848e+00	4.657931	0.03	3.32
altura_estipe	3116945.0	6.348333e+00	2.699755	0.00	4.67
largura_estipe	3116945.0	1.115379e+01	8.095477	0.00	4.97

	50%	75%	max
id	1558472.00	2337708.00	3116944.00
diametro_chapeu	5.75	8.24	80.67
altura_estipe	5.88	7.41	88.72
largura_estipe	9.65	15.63	102.90



2.5 Etapa 3: Pré-processamento dos Dados

2.5.1 Visão geral da execução anterior

Na etapa 2 realizamos a análise exploratória dos dados. Observamos que:

- A maioria das colunas são categóricas codificadas por letras, com muitos valores ausentes.

- Existem quatro colunas numéricas (`dimetro_chapeu`, `altura_estipe`, `largura_estipe`, `id`).
- A coluna alvo `classe` possui duas categorias (`c` e `v`).
- Há valores ausentes significativos em várias colunas, principalmente nas que representam características físicas.

2.5.2 Objetivo desta etapa

Preparar os DataFrames `train_df` e `test_df` para o treinamento do modelo, seguindo boas práticas de engenharia de dados:

1. **Criar cópias** dos DataFrames originais (`train_copy`, `test_copy`) para evitar vazamento de informação.
2. **Tratar valores ausentes**:
 - Para colunas numéricas: imputação com a mediana (robusta a outliers).
 - Para colunas categóricas: imputação com a string `'missing'` para preservar a cardinalidade.
3. **Codificar variáveis categóricas**:
 - Utilizar `LabelEncoder` de `sklearn.preprocessing`.
 - Ajustar o encoder em **todas as linhas combinadas** de treino e teste (concatenar as duas bases) para garantir que categorias presentes apenas no teste sejam reconhecidas.
 - Armazenar os encoders em um dicionário (`encoders`) para uso posterior na inferência.
4. **Mapear a coluna alvo classe** para valores numéricos (`c` $\rightarrow$ 0, `v` $\rightarrow$ 1) apenas no conjunto de treino.
5. **Manter a coluna id** inalterada, pois será necessária para a submissão.
6. **Separar as features** (`X_train`, `X_test`) e o alvo (`y_train`).

2.5.3 Por que não escrever arquivos nesta etapa?

Para evitar vazamento de informação e garantir que o notebook permaneça reproduzível, todas as transformações são realizadas em memória. Os DataFrames processados serão usados nas próximas etapas (divisão, treinamento, avaliação e geração de submissão).

2.5.4 Próximos passos

Com os dados já limpos e codificados, avançaremos para a divisão em treino/validação, treinamento do modelo base, avaliação com MCC e, finalmente, geração do arquivo `submission.csv` inicial.

(Esta célula contém apenas a documentação e a descrição do que será feito na etapa de pré-processamento. O código real será inserido em células de código subsequentes.)

```
[3]: # Etapa 3: Pré-processamento dos Dados (código)
import numpy as np
from sklearn.preprocessing import LabelEncoder

# 1. Criar cópias dos DataFrames originais
train_copy = train_df.copy()
test_copy = test_df.copy()
```

```

# 2. Identificar colunas numéricas e categóricas
numeric_cols = train_copy.select_dtypes(include=['number']).columns.tolist()
categorical_cols = train_copy.select_dtypes(include=['object']).columns.tolist()
# Remover a coluna alvo da lista de categóricas
if 'classe' in categorical_cols:
    categorical_cols.remove('classe')

# 3. Tratar valores ausentes
# - Numéricas: mediana
for col in numeric_cols:
    median_val = train_copy[col].median()
    train_copy[col].fillna(median_val, inplace=True)
    test_copy[col].fillna(median_val, inplace=True)
# - Categóricas: string 'missing'
for col in categorical_cols:
    train_copy[col].fillna('missing', inplace=True)
    test_copy[col].fillna('missing', inplace=True)

# 4. Codificar variáveis categóricas com LabelEncoder
encoders = {}
for col in categorical_cols:
    le = LabelEncoder()
    # Concatenar valores de treino e teste para garantir todas as categorias
    combined = pd.concat([train_copy[col], test_copy[col]], axis=0).astype(str)
    le.fit(combined)
    train_copy[col] = le.transform(train_copy[col].astype(str))
    test_copy[col] = le.transform(test_copy[col].astype(str))
    encoders[col] = le

# 5. Mapear a coluna alvo 'classe' para 0/1 (c -> 0, v -> 1)
train_copy['classe'] = train_copy['classe'].map({'c': 0, 'v': 1})

# 6. Separar features e alvo
X_train = train_copy.drop(columns=['classe'])
y_train = train_copy['classe']
X_test = test_copy.copy()

# 7. Exibir informações resumidas para verificação
print('Shape X_train:', X_train.shape)
print('Shape y_train:', y_train.shape)
print('Shape X_test:', X_test.shape)
print('\nTipos de dados após pré-processamento:')
print(X_train.dtypes)

```

Shape X_train: (3116945, 21)

Shape y_train: (3116945,)

Shape X_test: (2077964, 21)

Tipos de dados após pré-processamento:

id	int64
diametro_chapeu	float64
formato_chapeu	int64
superficie_chapeu	int64
cor_chapeu	int64
machuca_ou_sangra	int64
fixacao_lamela	int64
espacamento_lamela	int64
cor_lamela	int64
altura_estipe	float64
largura_estipe	float64
raiz_estipe	int64
superficie_estipe	int64
cor_estipe	int64
tipo_veu	int64
cor_veu	int64
tem_anel	int64
tipo_anel	int64
cor_impressao_esporo	int64
habitat	int64
estacao	int64

dtype: object

/tmp/ipykernel_83480/1693654615.py:20: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
train_copy[col].fillna(median_val, inplace=True)
```

/tmp/ipykernel_83480/1693654615.py:21: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
test_copy[col].fillna(median_val, inplace=True)
```

/tmp/ipykernel_83480/1693654615.py:20: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
train_copy[col].fillna(median_val, inplace=True)
```

/tmp/ipykernel_83480/1693654615.py:21: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
test_copy[col].fillna(median_val, inplace=True)
```

/tmp/ipykernel_83480/1693654615.py:20: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
train_copy[col].fillna(median_val, inplace=True)
```

/tmp/ipykernel_83480/1693654615.py:21: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing `'df[col].method(value, inplace=True)'`, try using `'df.method({col: value}, inplace=True)'` or `df[col] = df[col].method(value)` instead, to perform the operation inplace on the original object.

```
test_copy[col].fillna(median_val, inplace=True)
```

/tmp/ipykernel_83480/1693654615.py:20: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing `'df[col].method(value, inplace=True)'`, try using `'df.method({col: value}, inplace=True)'` or `df[col] = df[col].method(value)` instead, to perform the operation inplace on the original object.

```
train_copy[col].fillna(median_val, inplace=True)
```

/tmp/ipykernel_83480/1693654615.py:21: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing `'df[col].method(value, inplace=True)'`, try using `'df.method({col: value}, inplace=True)'` or `df[col] = df[col].method(value)` instead, to perform the operation inplace on the original object.

```
test_copy[col].fillna(median_val, inplace=True)
```

/tmp/ipykernel_83480/1693654615.py:24: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing `'df[col].method(value, inplace=True)'`, try using `'df.method({col: value}, inplace=True)'` or `df[col] = df[col].method(value)` instead, to perform the operation inplace on the original object.

```
train_copy[col].fillna('missing', inplace=True)
```

/tmp/ipykernel_83480/1693654615.py:25: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an

`inplace` method.

The behavior will change in pandas 3.0. This `inplace` method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing `'df[col].method(value, inplace=True)'`, try using `'df.method({col: value}, inplace=True)'` or `df[col] = df[col].method(value)` instead, to perform the operation `inplace` on the original object.

```
test_copy[col].fillna('missing', inplace=True)
```

2.6 Etapa 4: Divisão do Conjunto de Treino em Treino e Validação

2.6.1 Visão geral da execução anterior

1. **Carregamento:** os arquivos `train.csv` e `test.csv` foram lidos em `train_df` e `test_df`.
2. **EDA:** verificamos a distribuição das classes, cardinalidade das variáveis, valores ausentes e estatísticas resumidas das colunas numéricas.
3. **Pré-processamento:** criamos cópias `train_copy` e `test_copy`, imputamos valores ausentes (mediana para numéricas, `'missing'` para categóricas), codificamos todas as colunas categóricas com `LabelEncoder` (ajustado em treino + teste para garantir todas as categorias) e mapeamos a coluna alvo classe para 0/1.
4. **Separação de features e alvo:** obtivemos `X_train`, `y_train` e `X_test` prontos para o próximo passo.

2.6.2 Objetivo desta etapa

Dividir o conjunto de treino em duas partes: **80 %** para treinamento e **20 %** para validação. Essa divisão será feita de forma **estratificada** para manter a mesma proporção de classes em ambos os subconjuntos, garantindo que o modelo seja avaliado em dados representativos.

Passos que serão executados

1. **Importar** `train_test_split` de `sklearn.model_selection`.
2. **Definir** os parâmetros:
 - `test_size=0.2` (20 % do conjunto original).
 - `random_state=42` para reprodutibilidade.
 - `shuffle=True` (padrão) para garantir aleatoriedade.
 - `stratify=y_train` para preservar a distribuição de classes.
3. **Aplicar** a função, obtendo `X_tr`, `X_val`, `y_tr`, `y_val`.
4. **Verificar** as formas e a distribuição de classes nos subconjuntos para garantir que a estratificação funcionou.
5. **Manter** a coluna `id` inalterada em `X_tr` e `X_val` pois será necessária para a submissão futura.
6. **Não escrever** nenhum arquivo neste momento; todas as transformações permanecem em memória.

2.6.3 Por que essa divisão é importante

- **Validação interna:** permite medir o desempenho do modelo antes de treiná-lo em todo o conjunto, evitando overfitting.
- **Estratificação:** garante que a proporção de cogumelos comestíveis e venenosos seja a mesma em treino e validação, refletindo a realidade do problema.
- **Reprodutibilidade:** o `random_state` assegura que a mesma divisão seja obtida em execuções futuras.

2.6.4 Próximos passos

Com os subconjuntos prontos, avançaremos para a etapa de **treinamento do modelo base** (XGBoost ou CatBoost), seguida da **avaliação com MCC** e, finalmente, a **geração do arquivo `submission.csv` inicial**.

(Esta célula contém apenas a documentação e a descrição do que será feito na etapa de divisão. O código real será inserido em uma célula de código subsequente.)

```
[4]: from sklearn.model_selection import train_test_split

# Dividir em treino (80%) e validação (20%) com estratificação
X_tr, X_val, y_tr, y_val = train_test_split(
    X_train,
    y_train,
    test_size=0.2,
    random_state=42,
    stratify=y_train
)

# Verificar formas e distribuição de classes
print('Shape X_tr:', X_tr.shape)
print('Shape y_tr:', y_tr.shape)
print('Shape X_val:', X_val.shape)
print('Shape y_val:', y_val.shape)

print('\nDistribuição de classes no conjunto de treino:')
print(y_tr.value_counts(normalize=True))

print('\nDistribuição de classes no conjunto de validação:')
print(y_val.value_counts(normalize=True))
```

```
Shape X_tr: (2493556, 21)
Shape y_tr: (2493556,)
Shape X_val: (623389, 21)
Shape y_val: (623389,)
```

```
Distribuição de classes no conjunto de treino:
classe
```

```
1    0.547137
0    0.452863
Name: proportion, dtype: float64
```

Distribuição de classes no conjunto de validação:

```
classe
1    0.547137
0    0.452863
Name: proportion, dtype: float64
```

3 Etapa 5: Treinamento do Modelo Base

3.1 Visão geral da execução anterior

1. **Carregamento** dos arquivos `train.csv` e `test.csv` em `train_df` e `test_df`.
2. **EDA**: distribuição das classes, cardinalidade das variáveis, valores ausentes e estatísticas resumidas.
3. **Pré-processamento**: imputação (mediana para numéricas, 'missing' para categóricas), codificação com `LabelEncoder` (ajustado em treino + teste) e mapeamento da coluna alvo classe para 0/1.
4. **Divisão** em treino (80 %) e validação (20 %) estratificada, resultando em `X_tr`, `X_val`, `y_tr` e `y_val`.

3.2 Objetivo desta etapa

Treinar um **modelo base** que sirva como referência para as iterações de otimização posteriores. O modelo escolhido é o **XGBoost** (ou `CatBoost`, se preferir), pois combina alta performance em dados tabulares com facilidade de uso e suporte a multithreading.

3.2.1 Por que XGBoost?

- **Robustez**: lida bem com variáveis categóricas codificadas numericamente e com valores ausentes já tratados.
- **Velocidade**: com `n_estimators=500`, `max_depth=6`, `learning_rate=0.1` e `n_jobs=16` o treinamento costuma levar menos de 10 min em 128 GB de RAM.
- **Ajuste de hiperparâmetros**: os parâmetros padrão já produzem MCC acima de 0.9, o que já é competitivo para a competição.

3.2.2 Passos que serão executados

1. **Importar** `XGBClassifier` de `xgboost`.
2. **Instanciar** o modelo com os hiperparâmetros base:

```
model = XGBClassifier(
    n_estimators=500,
    max_depth=6,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
```

```

        objective='binary:logistic',
        eval_metric='logloss',
        n_jobs=16,
        random_state=42
    )

```

3. **Treinar** o modelo em `X_tr` e `y_tr`.
4. **Validar** usando `X_val` e `y_val`, calculando a métrica **Matthews Correlation Coefficient** (MCC) com `sklearn.metrics.matthews_corrcoef`.
5. **Armazenar** o modelo treinado em memória (ou em disco com `joblib.dump`) para uso posterior na geração de previsões.

3.2.3 Observações importantes

- O `n_jobs=16` garante que o XGBoost utilize até 16 threads, respeitando o limite de CPU.
- Como o conjunto de dados é grande, o treinamento pode levar alguns minutos; o notebook está configurado para não exceder 60 minutos.
- O MCC obtido nesta etapa servirá como baseline. Se for inferior ao esperado, iremos prosseguir para a etapa 9 (busca de hiperparâmetros) e depois treinar o modelo final.

3.3 Próximos passos

1. **Avaliar** o MCC e comparar com a métrica de referência da competição.
2. **Gerar** previsões binárias para o conjunto de teste usando o modelo base.
3. **Criar** o arquivo `submission.csv` inicial.
4. **Realizar** busca de hiperparâmetros (grid search ou Optuna) para melhorar o MCC.
5. **Re-treinar** o modelo final em todo o conjunto de treino e gerar a submissão final.

(Esta célula contém apenas a documentação e a descrição do que será feito na etapa de treinamento do modelo base. O código real será inserido em uma célula de código subsequente.)

```

[5]: from xgboost import XGBClassifier
    from sklearn.metrics import matthews_corrcoef

    # Instanciar o modelo XGBoost com parâmetros base
    base_model = XGBClassifier(
        n_estimators=500,
        max_depth=6,
        learning_rate=0.1,
        subsample=0.8,
        colsample_bytree=0.8,
        objective='binary:logistic',
        eval_metric='logloss',
        n_jobs=16,
        random_state=42,
        verbosity=0
    )

```

```
# Treinar no conjunto de treino
base_model.fit(X_tr, y_tr)

# Previsões no conjunto de validação
y_val_pred = base_model.predict(X_val)

# Calcular MCC
mcc_score = matthews_corrcoef(y_val, y_val_pred)
print('MCC no conjunto de validação:', mcc_score)
```

MCC no conjunto de validação: 0.9835840042892798

3.4 Etapa 6: Avaliação do Modelo Base

3.4.1 Visão geral da execução anterior

Na etapa 5 treinamos um **XGBoost** com hiperparâmetros padrão (500 estimadores, profundidade 6, taxa de aprendizado 0.1, etc.). O modelo foi ajustado em 80 % dos dados (**X_tr**, **y_tr**) e validado nos 20 % restantes (**X_val**, **y_val**).

3.4.2 Resultado da avaliação

O **Matthews Correlation Coefficient (MCC)** calculado no conjunto de validação foi **0.9836**, o que indica um desempenho excelente e já está muito próximo do limite de 1.0. Esse valor serve como baseline para as iterações de otimização que virão.

3.4.3 Próximos passos

1. **Gerar previsões binárias** para o conjunto de teste usando o modelo treinado (**base_model**).
2. **Criar o arquivo `submission.csv`** inicial com as previsões, no formato exigido pelo Kaggle (**id,classe**).
3. Se desejado, prosseguir com a busca de hiperparâmetros (tarefa 9) para tentar melhorar ainda mais o MCC, e então re-treinar em todo o conjunto de treino.

(Esta célula resume a etapa de avaliação e indica as próximas ações. O código para gerar as previsões e criar o arquivo de submissão será inserido em células de código subsequentes.)

```
[6]: from sklearn.metrics import matthews_corrcoef

# Calcular novamente o MCC no conjunto de validação
y_val_pred = base_model.predict(X_val)
mcc_score = matthews_corrcoef(y_val, y_val_pred)
print(f'MCC no conjunto de validação: {mcc_score:.6f}')
```

MCC no conjunto de validação: 0.983584

3.5 Etapa 7: Geração das Previsões para o Conjunto de Teste

3.5.1 Visão geral da execução anterior

- Treinamos um modelo XGBoost base com 500 estimadores, profundidade 6, etc.
- O MCC no conjunto de validação foi **0.9836**, indicando excelente desempenho.

3.5.2 Objetivo desta etapa

Gerar previsões binárias para o conjunto de teste usando o modelo já treinado (`base_model`).

3.5.3 Passos a serem realizados

1. **Aplicar o modelo:** `base_model.predict(X_test)` retorna 0/1.
2. **Mapear** os valores 0/1 de volta para as classes originais: $0 \rightarrow 'c'$, $1 \rightarrow 'v'$.
3. **Criar DataFrame** com duas colunas: `id` (do `test_df`) e `classe` (previsões mapeadas).
4. **Salvar** em `submission.csv` no diretório atual.

3.5.4 Observações

- Não alteramos a coluna `id` durante o pré-processamento, então ela está presente em `X_test`.
- Garantimos que o arquivo tenha cabeçalho `id,classe` e que cada linha contenha apenas a classe prevista.
- Este arquivo será a submissão inicial; caso façamos otimizações posteriores, o mesmo processo será repetido com o modelo final.

(Esta célula descreve a etapa de geração de previsões. O código correspondente será inserido em uma célula de código subsequente.)

```
[7]: import pandas as pd

# Gerar previsões binárias para o conjunto de teste
test_predictions = base_model.predict(X_test)

# Mapear 0/1 de volta para as classes originais ('c' e 'v')
pred_labels = pd.Series(test_predictions).map({0: 'c', 1: 'v'})

# Criar DataFrame de submissão com as colunas exigidas
submission_df = pd.DataFrame({
    'id': test_df['id'],
    'classe': pred_labels
})

# Salvar em arquivo CSV no formato correto
submission_df.to_csv('submission.csv', index=False)

# Exibir as primeiras linhas para verificação
print(submission_df.head())
```

	id	classe
0	3116945	c
1	3116946	v
2	3116947	v
3	3116948	v
4	3116949	c

4 Etapa 8: Criação do Arquivo de Submissão Inicial

4.1 Visão geral da execução anterior

1. **Carregamento** dos arquivos `train.csv` e `test.csv`.
2. **EDA**: distribuição das classes, cardinalidade e valores ausentes.
3. **Pré-processamento**: imputação, codificação com `LabelEncoder` e mapeamento da coluna alvo para 0/1.
4. **Divisão** em treino (80 %) e validação (20 %) estratificada.
5. **Treinamento** de um modelo XGBoost base e avaliação com MCC, obtendo **0.9836**.
6. **Previsões** para o conjunto de teste geradas pelo modelo base.

4.2 Objetivo desta etapa

Criar o arquivo `submission.csv` inicial, contendo as previsões binárias geradas no passo 7, no formato exigido pelo Kaggle: duas colunas, `id` e `classe`, onde `classe` deve ser `c` ou `v`.

4.3 Como o arquivo foi gerado

- O modelo `base_model` já estava treinado em `X_tr/y_tr`.
- Usamos `base_model.predict(X_test)` para obter 0/1.
- Mapeamos $0 \rightarrow c$ e $1 \rightarrow v$ com `pd.Series.map`.
- Construímos um `DataFrame` com as colunas `id` (do `test_df`) e `classe` (previsões mapeadas).
- Salvamos com `to_csv('submission.csv', index=False)`.

4.4 Próximos passos

1. Se desejar melhorar a performance, prosseguir com a busca de hiperparâmetros (tarefa 9).
2. Re-treinar o modelo final em todo o conjunto de treino e gerar uma nova submissão.
3. Submeter o arquivo `submission.csv` ao Kaggle para avaliação.

(Esta célula resume a etapa de criação do arquivo de submissão inicial e descreve o processo utilizado. O código já foi executado em células anteriores.)

```
[8]: # Geração do arquivo submission.csv inicial
import pandas as pd

# Previsões binárias para o conjunto de teste
test_predictions = base_model.predict(X_test)

# Mapear 0/1 de volta para as classes originais ('c' e 'v')
```

```

pred_labels = pd.Series(test_predictions).map({0: 'c', 1: 'v'})

# Criar DataFrame de submissão com as colunas exigidas
submission_df = pd.DataFrame({
    'id': test_df['id'],
    'classe': pred_labels
})

# Salvar em arquivo CSV no formato correto
submission_df.to_csv('submission.csv', index=False)

# Exibir as primeiras linhas para verificação
print(submission_df.head())

```

```

      id classe
0  3116945     c
1  3116946     v
2  3116947     v
3  3116948     v
4  3116949     c

```

5 Etapa 9: Busca de Hiperparâmetros

5.1 Visão geral da execução anterior

1. **Carregamento** dos dados e **pré-processamento** (imputação, codificação e mapeamento da classe). Os DataFrames `X_train`, `y_train`, `X_test` e `X_tr`, `X_val`, `y_tr`, `y_val` já foram criados.
2. **Treinamento do modelo base**: um `XGBClassifier` com 500 estimadores, profundidade 6 e taxa de aprendizado 0.1. O MCC no conjunto de validação foi **0.9836**, um desempenho excelente que serve como baseline.
3. **Geração da submissão inicial**: previsões binárias foram produzidas para o conjunto de teste e salvas em `submission.csv`.

5.2 Objetivo desta etapa

Melhorar ainda mais o MCC no conjunto de validação, explorando o espaço de hiperparâmetros do XGBoost. A ideia é encontrar um conjunto de parâmetros que maximize a métrica sem ultrapassar o limite de 60 minutos de execução e respeitando o uso de até 16 threads.

5.3 Estratégia de busca

1. **Optuna** será usado como framework de otimização. Ele permite definir um objetivo que treina o modelo e devolve o MCC, e então escolhe os melhores parâmetros com base em um algoritmo de otimização bayesiano.
2. **Parâmetros a explorar** (exemplo):
 - `n_estimators`: 200-800
 - `max_depth`: 4-10
 - `learning_rate`: 0.01-0.2

- `subsample`: 0.6-1.0
 - `colsample_bytree`: 0.6-1.0
 - `gamma`: 0-5
 - `min_child_weight`: 1-10
3. **Early stopping**: durante o treinamento, o XGBoost pode parar se a métrica de validação não melhorar em 50 rounds, evitando overfitting e economizando tempo.
 4. **Número de trials**: 30-50, dependendo do tempo disponível. Cada trial treina em `X_tr/y_tr` e avalia em `X_val/y_val`.
 5. **Parâmetros finais**: o conjunto que obtiver o maior MCC será salvo em `best_params` e usado na etapa 10 para re-treinar em todo o conjunto de treino.

5.4 Como proceder

- Definir a função objetivo do Optuna que recebe um `trial`, cria um `XGBClassifier` com os parâmetros sugeridos, treina com `early_stopping_rounds=50` e devolve o MCC.
- Executar `study.optimize(objective, n_trials=40, timeout=1800)` (30 minutos máximo) para garantir que a busca não ultrapasse o limite de tempo.
- Após a busca, imprimir os melhores parâmetros e o MCC correspondente.
- Guardar o objeto `study` ou apenas os parâmetros em variáveis globais para uso posterior.

5.5 Observações importantes

- **Reprodutibilidade**: definir `random_state=42` em todos os objetos que o aceitam.
- **Uso de CPU**: manter `n_jobs=16` no XGBoost.
- **Validação**: usar o mesmo split estratificado já criado (`X_tr`, `X_val`, `y_tr`, `y_val`).
- **Tempo**: se a busca exceder o limite, interrompa e use os melhores parâmetros encontrados até então.

5.6 Próximos passos

1. Executar a busca de hiperparâmetros (tarefa 9).
2. Re-treinar o modelo final em todo o conjunto de treino com os parâmetros ótimos (tarefa 10).
3. Gerar a submissão final (tarefa 12).

(Esta célula descreve a etapa de busca de hiperparâmetros. O código correspondente será inserido em uma célula de código subsequente.)

```
[9]: import optuna
from xgboost import XGBClassifier
from sklearn.metrics import matthews_corrcoef

def objective(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 200, 800),
        'max_depth': trial.suggest_int('max_depth', 4, 10),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2,
        log=True),
```



```

        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.6, 1.0),
        'gamma': trial.suggest_float('gamma', 0, 5),
        'min_child_weight': trial.suggest_int('min_child_weight', 1, 10),
        'objective': 'binary:logistic',
        'eval_metric': 'logloss',
        'n_jobs': 16,
        'random_state': 42,
        'verbosity': 0
    }
    model = XGBClassifier(**params)
    model.fit(
        X_tr, y_tr,
        eval_set=[(X_val, y_val)],
        early_stopping_rounds=50,
        verbose=False
    )
    preds = model.predict(X_val)
    mcc = matthews_corrcoef(y_val, preds)
    return mcc

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=30, timeout=1800)

print('Best trial:')
trial = study.best_trial
print(f'  MCC: {trial.value:.6f}')
print('  Params:')
for key, value in trial.params.items():
    print(f'    {key}: {value}')

best_params = trial.params

```

Best trial:

MCC: 0.984155

Params:

n_estimators: 581

max_depth: 10

learning_rate: 0.04533029713789278

subsample: 0.8386917764289885

colsample_bytree: 0.9459568200065331

gamma: 3.7765110713370165

min_child_weight: 6

[I 2025-12-21 03:06:08,735] A new study created in memory with name: no-name-190a0fc3-1489-4b97-b980-c37b96a09a0b

/Project/p3_11-venv/lib/python3.11/site-packages/xgboost/sklearn.py:861:

UserWarning: `early\_stopping\_rounds` in `fit` method is deprecated for better

```

compatibility with scikit-learn, use `early_stopping_rounds` in constructor
or `set_params` instead.
    warnings.warn(
[I 2025-12-21 03:08:39,535] Trial 0 finished with value: 0.9587363818069297 and
parameters: {'n_estimators': 339, 'max_depth': 4, 'learning_rate':
0.06668183796216537, 'subsample': 0.7253176002349278, 'colsample_bytree':
0.7375439427881922, 'gamma': 1.736155600369278, 'min_child_weight': 4}. Best is
trial 0 with value: 0.9587363818069297.
/Project/p3_11-venv/lib/python3.11/site-packages/xgboost/sklearn.py:861:
UserWarning: `early_stopping_rounds` in `fit` method is deprecated for better
compatibility with scikit-learn, use `early_stopping_rounds` in constructor
or `set_params` instead.
    warnings.warn(
[I 2025-12-21 03:10:53,518] Trial 1 finished with value: 0.9802839774981326 and
parameters: {'n_estimators': 300, 'max_depth': 4, 'learning_rate':
0.18963580871265726, 'subsample': 0.7234187233033154, 'colsample_bytree':
0.7190712998435234, 'gamma': 2.7420641250762716, 'min_child_weight': 2}. Best is
trial 1 with value: 0.9802839774981326.
/Project/p3_11-venv/lib/python3.11/site-packages/xgboost/sklearn.py:861:
UserWarning: `early_stopping_rounds` in `fit` method is deprecated for better
compatibility with scikit-learn, use `early_stopping_rounds` in constructor
or `set_params` instead.
    warnings.warn(
[I 2025-12-21 03:18:27,395] Trial 2 finished with value: 0.9783458189434292 and
parameters: {'n_estimators': 751, 'max_depth': 6, 'learning_rate':
0.023059265311772926, 'subsample': 0.990637558642195, 'colsample_bytree':
0.6028418092734261, 'gamma': 4.572336704765134, 'min_child_weight': 1}. Best is
trial 1 with value: 0.9802839774981326.
/Project/p3_11-venv/lib/python3.11/site-packages/xgboost/sklearn.py:861:
UserWarning: `early_stopping_rounds` in `fit` method is deprecated for better
compatibility with scikit-learn, use `early_stopping_rounds` in constructor
or `set_params` instead.
    warnings.warn(
[I 2025-12-21 03:22:14,153] Trial 3 finished with value: 0.9776047843476063 and
parameters: {'n_estimators': 329, 'max_depth': 6, 'learning_rate':
0.046881402411875134, 'subsample': 0.6251887355521927, 'colsample_bytree':
0.8644418835078459, 'gamma': 3.483253167522929, 'min_child_weight': 7}. Best is
trial 1 with value: 0.9802839774981326.
/Project/p3_11-venv/lib/python3.11/site-packages/xgboost/sklearn.py:861:
UserWarning: `early_stopping_rounds` in `fit` method is deprecated for better
compatibility with scikit-learn, use `early_stopping_rounds` in constructor
or `set_params` instead.
    warnings.warn(
[I 2025-12-21 03:34:07,330] Trial 4 finished with value: 0.9841548105572874 and
parameters: {'n_estimators': 581, 'max_depth': 10, 'learning_rate':
0.04533029713789278, 'subsample': 0.8386917764289885, 'colsample_bytree':
0.9459568200065331, 'gamma': 3.7765110713370165, 'min_child_weight': 6}. Best is
trial 4 with value: 0.9841548105572874.

```

```
/Project/p3_11-venv/lib/python3.11/site-packages/xgboost/sklearn.py:861:
UserWarning: `early_stopping_rounds` in `fit` method is deprecated for better
compatibility with scikit-learn, use `early_stopping_rounds` in constructor
or `set_params` instead.
  warnings.warn(
[I 2025-12-21 03:38:57,116] Trial 5 finished with value: 0.984131943343695 and
parameters: {'n_estimators': 301, 'max_depth': 9, 'learning_rate':
0.16814863516987366, 'subsample': 0.8496946290398384, 'colsample_bytree':
0.7571966284691956, 'gamma': 4.583032151760687, 'min_child_weight': 10}. Best is
trial 4 with value: 0.9841548105572874.
```

6 Etapa 10: Re-treinamento do Modelo Final

6.1 Visão geral da execução anterior

1. **Busca de hiperparâmetros** (tarefa 9) utilizou o Optuna para otimizar um `XGBClassifier`. O melhor conjunto de parâmetros encontrado foi:

```
n_estimators: 581
max_depth: 10
learning_rate: 0.0453
subsample: 0.8387
colsample_bytree: 0.9460
gamma: 3.7765
min_child_weight: 6
```

O MCC no conjunto de validação com esses parâmetros foi **0.98415**, um pequeno ganho em relação ao modelo base (0.98358).

6.2 Objetivo desta etapa

Treinar o modelo final **em todo o conjunto de treino** (`X_train`, `y_train`) usando os hiperparâmetros ótimos encontrados. Em seguida, gerar previsões para o conjunto de teste, mapear 0/1 de volta para c/v e criar o arquivo `submission.csv` final, pronto para upload no Kaggle.

6.3 Passos a serem realizados

1. **Instanciar** o `XGBClassifier` com os parâmetros `best_params` e `n_jobs=16`.
2. **Treinar** o modelo em `X_train/y_train`. Como o conjunto completo já contém 3,116,945 observações, o treinamento pode levar alguns minutos, mas permanece dentro do limite de 60 minutos.
3. **Prever** as classes binárias para `X_test`.
4. **Mapear** $0 \rightarrow c$ e $1 \rightarrow v$.
5. **Criar** o DataFrame de submissão com as colunas `id` (do `test_df`) e `classe` (previsões mapeadas).
6. **Salvar** em `submission.csv` usando `to_csv(index=False)`.
7. **Verificar** as primeiras linhas do arquivo para garantir o formato correto.

6.4 Observações importantes

- O modelo final será treinado **sem** validação hold-out, pois queremos aproveitar todo o poder de aprendizado disponível.
- Se desejar manter a possibilidade de re-treinar com outro conjunto de parâmetros, basta repetir o processo com os novos valores.
- O arquivo `submission.csv` gerado nesta etapa substituirá o arquivo criado na etapa 8, garantindo que a submissão enviada ao Kaggle contenha as previsões mais precisas.

6.5 Próximos passos

1. **Executar** a célula de código que treina o modelo final e gera a submissão.
2. **Submeter** o arquivo `submission.csv` ao Kaggle para avaliação.
3. Se necessário, repetir a busca de hiperparâmetros (tarefa 9) e re-treinar novamente.

(Esta célula resume a etapa de re-treinamento do modelo final e descreve os passos que serão executados em código nas células subsequentes.)

```
[10]: # Etapa 10: Re-treinamento do Modelo Final e Geração da Submissão Final

# Importar o XGBClassifier (já importado anteriormente, mas reimportar para
↳ garantir)
from xgboost import XGBClassifier

# Combinar os parâmetros ótimos com os parâmetros fixos necessários
final_params = {
    "objective": "binary:logistic",
    "eval_metric": "logloss",
    "n_jobs": 16,
    "random_state": 42,
    "verbosity": 0
}
final_params.update(best_params) # adiciona os hiperparâmetros otimizados

# Instanciar o modelo final
final_model = XGBClassifier(**final_params)

# Treinar no conjunto completo de treino
final_model.fit(X_train, y_train)

# Previsões no conjunto de teste
test_pred = final_model.predict(X_test)

# Mapear 0/1 de volta para 'c' e 'v'
pred_labels = pd.Series(test_pred).map({0: "c", 1: "v"})

# Criar DataFrame de submissão
```

```

submission_df = pd.DataFrame({
    "id": test_df["id"],
    "classe": pred_labels
})

# Salvar em arquivo CSV
submission_df.to_csv("submission.csv", index=False)

# Exibir as primeiras linhas para verificação
print(submission_df.head())

```

Cell In[10], line 4

```

from xgboost import XGBClassifier
^

```

IndentationError: unexpected indent

```

[11]: from xgboost import XGBClassifier
import pandas as pd

# Combinar os parâmetros ótimos com os parâmetros fixos necessários
final_params = {
    "objective": "binary:logistic",
    "eval_metric": "logloss",
    "n_jobs": 16,
    "random_state": 42,
    "verbosity": 0
}
final_params.update(best_params) # adiciona os hiperparâmetros otimizados

# Instanciar o modelo final
final_model = XGBClassifier(**final_params)

# Treinar no conjunto completo de treino
final_model.fit(X_train, y_train)

# Previsões no conjunto de teste
test_pred = final_model.predict(X_test)

# Mapear 0/1 de volta para 'c' e 'v'
pred_labels = pd.Series(test_pred).map({0: 'c', 1: 'v'})

# Criar DataFrame de submissão
submission_df = pd.DataFrame({
    'id': test_df['id'],
    'classe': pred_labels
})

```

```

})

# Salvar em arquivo CSV
submission_df.to_csv('submission.csv', index=False)

# Exibir as primeiras linhas para verificação
print(submission_df.head())

```

```

      id classe
0  3116945     c
1  3116946     v
2  3116947     v
3  3116948     v
4  3116949     c

```

6.6 Etapa 11: Geração das Previsões Finais

6.6.1 Visão geral da execução anterior

- **Carregamento e EDA:** os arquivos `train.csv` e `test.csv` foram lidos, analisados e as estatísticas básicas foram calculadas.
- **Pré-processamento:** valores ausentes foram imputados (mediana para numéricas, 'missing' para categóricas) e todas as colunas categóricas foram codificadas com `LabelEncoder`. A coluna alvo `classe` foi mapeada para 0/1.
- **Divisão:** o conjunto de treino foi separado em 80 % treino e 20 % validação estratificada.
- **Modelo base:** um `XGBClassifier` com 500 estimadores, profundidade 6 e taxa de aprendizado 0.1 foi treinado, obtendo MCC **0.9836** no conjunto de validação.
- **Busca de hiperparâmetros:** Optuna explorou o espaço de parâmetros e encontrou o melhor conjunto com MCC **0.98415** (`n_estimators=581`, `max_depth=10`, `learning_rate=0.0453`, `subsample=0.839`, `colsample_bytree=0.946`, `gamma=3.78`, `min_child_weight=6`).
- **Re-treinamento final:** o modelo foi treinado em todo o conjunto de treino (`X_train`, `y_train`) usando os parâmetros ótimos. As previsões foram geradas para `X_test`, mapeadas de volta para 'c'/'v' e salvas em `submission.csv`.

6.6.2 Passo atual

As previsões finais já foram geradas e o arquivo `submission.csv` foi criado no diretório de trabalho. Se quiser verificar a qualidade das previsões, basta abrir o arquivo ou submeter ao Kaggle. Caso deseje refinar ainda mais o modelo, pode repetir a busca de hiperparâmetros ou experimentar outro algoritmo.

6.6.3 Próximos passos

- **Submeter** o arquivo `submission.csv` ao Kaggle e acompanhar a pontuação no leaderboard.
- **Monitorar** a performance e, se necessário, iterar com novas feature engineering ou modelos alternativos.
- **Documentar** os resultados e salvar o notebook para referência futura.

```
[12]: # Gerar previsões finais para o conjunto de teste usando o modelo final treinado
final_predictions = final_model.predict(X_test)

# Mapear 0/1 de volta para as classes originais ('c' e 'v')
final_pred_labels = pd.Series(final_predictions).map({0: 'c', 1: 'v'})

# Exibir as primeiras previsões para verificação
print(final_pred_labels.head())
```

```
0    c
1    v
2    v
3    v
4    c
dtype: object
```

6.7 Etapa 12: Salvar Submissão Final

6.7.1 Visão geral da execução anterior

1. **Carregamento e EDA:** os arquivos `train.csv` e `test.csv` foram lidos e analisados.
2. **Pré-processamento:** valores ausentes foram imputados, variáveis categóricas codificadas com `LabelEncoder` e a coluna alvo `classe` mapeada para 0/1.
3. **Divisão:** 80 % treino / 20 % validação estratificada.
4. **Modelo base:** XGBoost com 500 estimadores, obtendo `MCC 0.9836`.
5. **Busca de hiperparâmetros:** Optuna encontrou o melhor conjunto de parâmetros (`n_estimators=581`, `max_depth=10`, etc.) com `MCC 0.98415`.
6. **Re-treinamento final:** o modelo foi treinado em todo o conjunto de treino (`X_train`, `y_train`) usando os parâmetros ótimos e gerou previsões para `X_test`.
7. **Submissão:** as previsões foram mapeadas de volta para c/v e salvas em `submission.csv`.

6.7.2 Passo atual

O arquivo `submission.csv` já foi criado no diretório de trabalho e contém as previsões finais no formato exigido pelo Kaggle (`id, classe`). Para garantir que tudo está correto, basta abrir o arquivo ou executar:

```
head -n 5 submission.csv
```

Se o conteúdo estiver no formato esperado, você pode submeter o arquivo ao Kaggle. Caso queira re-gerar a submissão (por exemplo, após ajustes no modelo), basta re-executar a célula de código que salva o arquivo.

(Esta célula resume a etapa final de salvamento da submissão e confirma que o arquivo está pronto para upload.)

```
[13]: # Salvar a submissão final (passo 12)
# Utiliza as previsões geradas pelo modelo final (final_predictions) e
# garante que o arquivo 'submission.csv' esteja pronto para upload no Kaggle.
```

```

import pandas as pd

# Mapear 0/1 de volta para as classes originais ('c' e 'v')
final_pred_labels = pd.Series(final_predictions).map({0: 'c', 1: 'v'})

# Criar DataFrame de submissão
submission_df = pd.DataFrame({
    'id': test_df['id'],
    'classe': final_pred_labels
})

# Salvar em arquivo CSV no formato correto
submission_df.to_csv('submission.csv', index=False)

# Exibir as primeiras linhas para verificação
print(submission_df.head())

```

	id	classe
0	3116945	c
1	3116946	v
2	3116947	v
3	3116948	v
4	3116949	c